



ShopEase™ Advanced E-Commerce DBMS – Executive Summary

ShopEase Advanced is a comprehensive e-commerce database solution built with Microsoft SQL Server (SSMS). It provides end-to-end management of online retail operations: products, customers, orders, inventory, and analytics. This report details the project vision, scope, and a rigorous design. We present a fully normalized schema with ER diagram, complete SQL DDL (tables with keys, indexes, constraints) and sample data, essential T-SQL modules (12 stored procedures, views, triggers, functions with purpose and code), and 10-12 key queries for reporting and operations. We also cover performance tuning (indexes, partitioning, columnstore, Query Store), backup/restore and security (TDE, Always Encrypted, Row-Level Security, roles), ETL via SSIS, SSRS reporting suggestions, testing/validation, a 10-week timeline (Mermaid diagram), and presentation outline. Sources include Microsoft docs (AdventureWorks sample DB, SQL Server guides) and reputable data sets (UCI/Kaggle).

Project Name & Branding

Name: ShopEase™ Advanced DBMS

Tagline: *"Empowering Online Retail with a Scalable, Secure SQL Server Backend."*

Objective and Scope

Objective: To build a robust, scalable SQL Server database that supports all functions of a modern e-commerce platform: product catalog, customer management, order processing, inventory tracking, and real-time analytics. The system must ensure data integrity, high performance under load, and secure handling of sensitive data (orders, customer info, payment details).

Scope: The project covers database design and implementation only (no front-end). It assumes existing or external web/app interfaces. Key features include: product and category hierarchy, detailed order/invoice processing, stock management, sales analytics (e.g. daily revenue, top products), and admin dashboards. Additional scope includes: data import via SSIS, reporting via SSRS, and best practices for indexing, partitioning, and security. Areas *not* covered: application-level logic beyond stored routines, advanced analytics beyond SQL reporting, and machine learning.

Logical Schema and ER Diagram

The database is normalized (up to 3NF) to eliminate redundancy. Main entities include **Product**, **Category**, **Customer**, **Order** (header), **OrderItem**, **Inventory**, and **Supplier**. Key relationships:

- **Product-Category:** Many-to-one (each product belongs to one category).
- **Order-Customer:** Many-to-one (each order is placed by one customer).
- **Order-OrderItem:** One-to-many (each order has multiple line items).
- **OrderItem-Product:** Many-to-one (each line item is for one product).

- **Inventory-Product:** One-to-one (each product has one inventory record).
- **Supplier-Product:** One-to-many (each product has one supplier).

The Mermaid ER diagram code below illustrates this structure:

```
erDiagram
    CATEGORY {
        int CategoryID PK
        varchar Name
        varchar Description
    }
    SUPPLIER {
        int SupplierID PK
        varchar Name
        varchar ContactInfo
    }
    PRODUCT {
        int ProductID PK
        varchar SKU
        varchar Name
        decimal Price
        int CategoryID FK
        int SupplierID FK
    }
    CUSTOMER {
        int CustomerID PK
        varchar Name
        varchar Email
        varchar Phone
    }
    "ORDER" {
        int OrderID PK
        int CustomerID FK
        datetime OrderDate
        decimal TotalAmount
    }
    ORDERITEM {
        int OrderItemID PK
        int OrderID FK
        int ProductID FK
        int Quantity
        decimal UnitPrice
    }
    INVENTORY {
        int InventoryID PK
        int ProductID FK
        int QuantityOnHand
    }
```

```

}
CATEGORY ||--o{ PRODUCT : categorizes
SUPPLIER ||--o{ PRODUCT : supplies
PRODUCT ||--o{ ORDERITEM : included_in
"ORDER" ||--o{ ORDERITEM : contains
CUSTOMER ||--o{ "ORDER" : places
PRODUCT ||--o{ INVENTORY : stock_for

```

Each box lists key attributes (PK=primary key, FK=foreign key). For example, **PRODUCT** has `CategoryID` referencing **CATEGORY** and `SupplierID` referencing **SUPPLIER**, enforcing referential integrity.

SQL DDL (Tables, Keys, Indexes) and Sample Data

Tables and Constraints: The SQL DDL below creates all tables with data types, primary/foreign keys, and indexes. For brevity, NOT NULL and defaults are omitted but should be applied (e.g. `NOT NULL` on required fields). Indexes beyond primary keys are listed after each table.

```

CREATE TABLE Category (
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,
    Description VARCHAR(250)
);

CREATE TABLE Supplier (
    SupplierID INT IDENTITY(1,1) PRIMARY KEY,
    Name       VARCHAR(150) NOT NULL,
    ContactInfo VARCHAR(250)
);

CREATE TABLE Product (
    ProductID INT IDENTITY(1,1) PRIMARY KEY,
    SKU       VARCHAR(50) NOT NULL UNIQUE,
    Name      VARCHAR(150) NOT NULL,
    Price     DECIMAL(10,2) NOT NULL,
    CategoryID INT,
    SupplierID INT,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);

-- Nonclustered indexes on foreign keys for faster joins
CREATE INDEX IX_Product_Category ON Product(CategoryID);
CREATE INDEX IX_Product_Supplier ON Product(SupplierID);

CREATE TABLE Customer (
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,

```

```

        Email          VARCHAR(100) NOT NULL UNIQUE,
        Phone          VARCHAR(20)
    );

CREATE TABLE [Order] (
    OrderID            INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID         INT,
    OrderDate          DATETIME NOT NULL,
    TotalAmount        DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
CREATE INDEX IX_Order_Customer ON [Order](CustomerID);
CREATE INDEX IX_Order_OrderDate ON [Order](OrderDate);

CREATE TABLE OrderItem (
    OrderItemID        INT IDENTITY(1,1) PRIMARY KEY,
    OrderID            INT,
    ProductID          INT,
    Quantity           INT NOT NULL,
    UnitPrice          DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES [Order](OrderID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
CREATE INDEX IX_OrderItem_Order ON OrderItem(OrderID);
CREATE INDEX IX_OrderItem_Product ON OrderItem(ProductID);

CREATE TABLE Inventory (
    InventoryID        INT IDENTITY(1,1) PRIMARY KEY,
    ProductID          INT UNIQUE,
    QuantityOnHand     INT NOT NULL,
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
CREATE INDEX IX_Inventory_Product ON Inventory(ProductID);

```

Sample Data: Below are **INSERT** statements with 10–15 rows for main tables. These illustrate possible values; in practice, data can be bulk-loaded from real sources. For brevity, only key columns are shown.

```

-- Categories
INSERT INTO Category (Name, Description) VALUES
('Electronics', 'Phones, TVs, computers'),
('Books', 'Fiction and non-fiction'),
('Home & Kitchen', 'Home appliances and decor');

-- Suppliers
INSERT INTO Supplier (Name, ContactInfo) VALUES
('Acme Supplies', 'acme@example.com'),

```

```

('Global Books','contact@globalbooks.com');

-- Products
INSERT INTO Product (SKU, Name, Price, CategoryID, SupplierID) VALUES
('ELEC-001','Smartphone Model X',699.00, 1, 1),
('ELEC-002','42-inch LED TV',399.00, 1, 1),
('BOOK-001','C# Programming Guide',49.99, 2, 2),
('HOME-001','Blender 500W',89.50,3,1),
('ELEC-003','Wireless Headphones','199.00',1,1);

-- Customers
INSERT INTO Customer (Name, Email, Phone) VALUES
('Alice Johnson','alice@mail.com','555-1234'),
('Bob Smith','bob@example.com','555-2345'),
('Charlie Lee','charlie@domain.com','555-3456');

-- Orders
INSERT INTO [Order] (CustomerID, OrderDate, TotalAmount) VALUES
(1,'2026-01-10 10:15', 748.99),
(2,'2026-01-11 14:30', 449.00),
(1,'2026-01-12 16:45', 49.99),
(3,'2026-01-13 09:20', 288.50);

-- OrderItems
INSERT INTO OrderItem (OrderID, ProductID, Quantity, UnitPrice) VALUES
(1, 1, 1, 699.00), -- Alice buys Smartphone
(1, 4, 1, 49.99), -- Alice also buys C# book
(2, 2, 1, 399.00), -- Bob buys TV
(2, 5, 1, 49.00), -- Bob buys headphones (discounted)
(3, 3, 1, 49.99), -- Alice buys book
(4, 1, 1, 699.00), -- Charlie buys Smartphone
(4, 4, 1, 49.99); -- Charlie buys book

-- Inventory
INSERT INTO Inventory (ProductID, QuantityOnHand) VALUES
(1, 50),(2, 20),(3,100),(4,30),(5,75);

```

Each table is illustrated with some rows (e.g. **Order** and **OrderItem** show two orders with two items each). This sample data enables testing of constraints and queries.

Essential T-SQL Modules (Stored Procs, Views, etc.)

Below are 12 key database modules and their purpose, each with example code snippets. These cover typical e-commerce operations and reporting.

1. `sp_CreateOrder` (**Stored Procedure**) – Inserts a new order with its items atomically. It accepts CustomerID and a table-valued parameter of items (ProductID,Quantity). It validates stock, inserts

into `Order` and `OrderItem`, and updates **Inventory**.

```
CREATE PROCEDURE sp_CreateOrder
    @CustomerID INT,
    @Items dbo.OrderItemsType READONLY -- custom TVP (ProductID, Quantity)
AS
BEGIN
    DECLARE @NewOrderID INT;
    BEGIN TRAN;
    INSERT INTO [Order] (CustomerID, OrderDate, TotalAmount)
    VALUES (@CustomerID, GETDATE(), 0);
    SET @NewOrderID = SCOPE_IDENTITY();

    DECLARE @total DECIMAL(12,2) = 0;
    -- Loop through items (simplified)
    INSERT INTO OrderItem (OrderID, ProductID, Quantity, UnitPrice)
    SELECT @NewOrderID, ProductID, Quantity, Price
    FROM @Items JOIN Product ON @Items.ProductID = Product.ProductID;

    -- Update total and inventory
    SELECT @total = SUM(Quantity * UnitPrice) FROM OrderItem WHERE
OrderID=@NewOrderID;
    UPDATE [Order] SET TotalAmount=@total WHERE OrderID=@NewOrderID;
    UPDATE inv
    SET inv.QuantityOnHand = inv.QuantityOnHand - oi.Quantity
    FROM Inventory inv JOIN OrderItem oi ON inv.ProductID = oi.ProductID
    WHERE oi.OrderID = @NewOrderID;

    COMMIT;
END;
```

2. `trg_Order_Insert` (Trigger) - AFTER INSERT on **OrderItem**. Verifies stock levels: if **Inventory.QuantityOnHand** becomes negative, rolls back and raises an error.

```
CREATE TRIGGER trg_OrderItem_CheckStock
ON OrderItem
AFTER INSERT
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Inventory inv
        JOIN inserted i ON inv.ProductID = i.ProductID
        WHERE inv.QuantityOnHand - i.Quantity < 0
    )
```

```

BEGIN
    RAISERROR('Insufficient stock for an inserted order item.', 16, 1);
    ROLLBACK TRAN;
END
END;

```

3. **vw_DailySales** (**View**) – Summarizes total sales per day. Useful for reporting daily revenue.

```

CREATE VIEW vw_DailySales AS
SELECT CAST(OrderDate AS DATE) AS SaleDate,
       SUM(TotalAmount) AS TotalSales
FROM [Order]
GROUP BY CAST(OrderDate AS DATE);

```

4. **fn_CustomerOrderCount** (**Scalar Function**) – Returns the count of orders for a given CustomerID.

```

CREATE FUNCTION fn_CustomerOrderCount(@CustID INT)
RETURNS INT
AS
BEGIN
    DECLARE @cnt INT;
    SELECT @cnt = COUNT(*) FROM [Order] WHERE CustomerID = @CustID;
    RETURN @cnt;
END;

```

5. **trg_Order_UpdateTotal** (**Trigger**) – AFTER INSERT or UPDATE on **OrderItem**. Re-calculates the parent order's TotalAmount when items change.

```

CREATE TRIGGER trg_UpdateOrderTotal
ON OrderItem
AFTER INSERT, UPDATE
AS
BEGIN
    DECLARE @oid INT;
    SELECT @oid = OrderID FROM inserted;
    UPDATE o
    SET TotalAmount = (SELECT SUM(Quantity * UnitPrice) FROM OrderItem
    WHERE OrderID = @oid)
    FROM [Order] o WHERE o.OrderID = @oid;
END;

```

6. `vw_ProductInventory` (**View**) – Lists products with current stock, using JOIN on **Inventory**.

```
CREATE VIEW vw_ProductInventory AS
SELECT p.ProductID, p.Name, inv.QuantityOnHand
FROM Product p
JOIN Inventory inv ON p.ProductID = inv.ProductID;
```

7. `sp_UpdatePrice` (**Stored Procedure**) – Batch-update prices (e.g. apply discount). Demonstrates a data update proc.

```
CREATE PROCEDURE sp_UpdatePrice
    @ProductID INT,
    @NewPrice DECIMAL(10,2)
AS
BEGIN
    UPDATE Product
    SET Price = @NewPrice
    WHERE ProductID = @ProductID;
END;
```

8. `trg_Inventory_Insert` (**Trigger**) – BEFORE INSERT on **Inventory** (implemented as an INSTEAD OF trigger), to set default quantity 0 if missing.

```
CREATE TRIGGER trg_SetDefaultInventory ON Inventory
INSTEAD OF INSERT
AS
BEGIN
    INSERT INTO Inventory (ProductID, QuantityOnHand)
    SELECT COALESCE(ProductID, 0), COALESCE(QuantityOnHand, 0)
    FROM inserted;
END;
```

9. `fn_TotalStockValue` (**Scalar Function**) – Calculates total value of inventory (for analytics).

```
CREATE FUNCTION fn_TotalStockValue()
RETURNS DECIMAL(18,2)
AS
BEGIN
    RETURN (SELECT SUM(p.Price * inv.QuantityOnHand)
            FROM Product p JOIN Inventory inv ON
```



```
p.ProductID=inv.ProductID);
END;
```

10. `sp_PlaceOrderValidated` (**Stored Procedure**) – Wrapper around `sp_CreateOrder` with checks (e.g. credit limit, etc.) – pseudocode omitted.

```
-- Example: ensures customer has positive balance before placing order
CREATE PROCEDURE sp_PlaceOrderValidated
    @CustomerID INT,
    @Items dbo.OrderItemsType READONLY
AS
BEGIN
    -- Validate business rules (not shown)
    EXEC sp_CreateOrder @CustomerID, @Items;
END;
```

11. `trg_CancelOrder` (**Trigger**) – ON UPDATE of **Order** to handle cancellations. If `Order.TotalAmount` set to 0 or marked cancelled, replenishes inventory.

```
CREATE TRIGGER trg_HandleOrderCancel
ON [Order]
AFTER UPDATE
AS
BEGIN
    IF UPDATE(TotalAmount)
    BEGIN
        -- If TotalAmount becomes 0, assume cancellation: restock items
        IF EXISTS(SELECT 1 FROM deleted d JOIN inserted i ON
d.OrderID=i.OrderID
                WHERE d.TotalAmount>0 AND i.TotalAmount=0)
        BEGIN
            -- Restock inventory
            UPDATE inv
            SET inv.QuantityOnHand = inv.QuantityOnHand + oi.Quantity
            FROM Inventory inv
            JOIN OrderItem oi ON inv.ProductID = oi.ProductID
            JOIN inserted i ON oi.OrderID = i.OrderID
            WHERE i.TotalAmount = 0;
        END
    END
END;
```

12. `vw_TopProducts` (**View**) – Shows top 5 selling products by quantity.

```
CREATE VIEW vw_TopProducts AS
SELECT TOP 5 p.ProductID, p.Name, SUM(oi.Quantity) AS UnitsSold
FROM OrderItem oi
JOIN Product p ON oi.ProductID = p.ProductID
GROUP BY p.ProductID, p.Name
ORDER BY UnitsSold DESC;
```

Each module is crucial: triggers maintain data consistency, procedures encapsulate business logic, and views/functions support reporting.

Representative Queries (Analytics & Ops)

Below are 10 sample queries with purpose and SQL. These demonstrate reporting and operational needs:

1. **Daily Sales Report:** Show total sales per day (for SSRS).

```
SELECT CAST(OrderDate AS DATE) AS SaleDate, SUM(TotalAmount) AS TotalSales
FROM [Order]
GROUP BY CAST(OrderDate AS DATE)
ORDER BY SaleDate;
```

2. **Customer Order History:** List orders for a given customer.

```
SELECT o.OrderID, o.OrderDate, o.TotalAmount
FROM [Order] o
WHERE o.CustomerID = 1
ORDER BY o.OrderDate DESC;
```

3. **Top Products by Revenue:** Sum revenue per product, top 5.

```
SELECT TOP 5 p.ProductID, p.Name, SUM(oi.Quantity * oi.UnitPrice) AS
Revenue
FROM OrderItem oi
JOIN Product p ON oi.ProductID = p.ProductID
GROUP BY p.ProductID, p.Name
ORDER BY Revenue DESC;
```

4. **Low Stock Alert:** Products with inventory below threshold.

```
SELECT p.ProductID, p.Name, inv.QuantityOnHand
FROM Product p
JOIN Inventory inv ON p.ProductID = inv.ProductID
WHERE inv.QuantityOnHand < 10;
```

5. **Monthly Sales Trend:** Sales totals by month (uses GROUP BY and a window function).

```
SELECT
    YEAR(OrderDate) AS Yr,
    MONTH(OrderDate) AS Mo,
    SUM(TotalAmount) AS MonthlySales,
    SUM(SUM(TotalAmount)) OVER (ORDER BY YEAR(OrderDate), MONTH(OrderDate))
AS CumulativeSales
FROM [Order]
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY Yr, Mo;
```

6. **Customer Lifetime Value:** Total spent by each customer.

```
SELECT c.CustomerID, c.Name, SUM(o.TotalAmount) AS LifetimeSpend
FROM Customer c
JOIN [Order] o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
ORDER BY LifetimeSpend DESC;
```

7. **Product Search (Parameterized):** Find products by name substring.

```
SELECT ProductID, Name, Price
FROM Product
WHERE Name LIKE '%phone%'
ORDER BY Name;
```

8. **Inventory Replenishment Needs:** List products needing reorder (QuantityOnHand below average).

```
SELECT p.ProductID, p.Name, inv.QuantityOnHand
FROM Inventory inv
JOIN Product p ON inv.ProductID = p.ProductID
WHERE inv.QuantityOnHand < (
    SELECT AVG(QuantityOnHand) FROM Inventory);
```

9. **Cancelled Orders:** Identify orders where TotalAmount = 0 (via trigger cancellation).

```
SELECT OrderID, CustomerID, OrderDate
FROM [Order]
WHERE TotalAmount = 0;
```

10. **Sales by Category:** Total revenue by product category.

```
SELECT c.CategoryID, c.Name, SUM(oi.Quantity * oi.UnitPrice) AS Revenue
FROM OrderItem oi
JOIN Product p ON oi.ProductID = p.ProductID
JOIN Category c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryID, c.Name;
```

Each query includes a clear purpose (reporting, analytics, or operational check) and demonstrates JOINS, aggregations, and filtering relevant to an e-commerce domain.

Indexing & Performance Tuning

Efficient indexing is critical: “**Designing efficient indexes is key to good database performance**” ¹. Our strategy:

- **Clustered Indexes:** Each PK above is clustered by default, organizing data physically by ID.
- **Nonclustered Indexes:** As shown in DDL, we add indexes on foreign keys (e.g. `IX_Order_Customer`) and on commonly filtered columns (`IX_Order_OrderDate` for date-range queries).

- **Columnstore Index:** For the large **OrderItem** fact table, consider a **clustered columnstore index** (analytics mode). Columnstore yields ~10× data compression compared to rowstore ² and dramatically speeds aggregate queries. E.g., after heavy initial loading, run:

```
CREATE CLUSTERED COLUMNSTORE INDEX cci_OrderItem ON OrderItem;
```

- **Filtered Indexes:** For sparse queries (e.g. active orders), filtered indexes improve performance. Example:

```
CREATE INDEX IX_Order_Active ON [Order](OrderDate) WHERE TotalAmount > 0;
```

- **Partitioning:** (SQL Server Enterprise) Partition large tables by date. For example, a partition function on Year(OrderDate) splits `Order` into yearly partitions. This improves manageability (switch out old data) and can speed queries that target recent years.

- **Statistics and Query Store:** Enable **AUTO_UPDATE_STATISTICS**. Use the **Query Store** (SQL Server 2016+) to track query performance and force plans if necessary. Regularly update statistics on large tables after bulk loads.
- **Maintenance:** Rebuild indexes during off-hours. Consider **online index rebuilds** for minimal downtime. Use the Database Engine Tuning Advisor and SQL Profiler to spot missing indexes.

By following these strategies and leveraging columnstore indexing for analytics-heavy tables ², we ensure fast query execution even on large datasets.

Backup/Restore and Security

- **Backups:** Use SQL Server full and transaction log backups. Test restores: e.g., in SSMS or via T-SQL `RESTORE DATABASE`. Ensure off-site/offline backups.
- **Transparent Data Encryption (TDE):** Protects data-at-rest (encrypts data and log files) ³. Enable TDE on ShopEase databases so stolen backups or files cannot be read without keys. Example steps: create a master key, certificate, then:

```
USE ShopEaseDB;
CREATE DATABASE ENCRYPTION KEY WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE MyTDECert;
ALTER DATABASE ShopEaseDB SET ENCRYPTION ON;
```

Citations: TDE performs real-time I/O encryption of data/log files ³.

- **Always Encrypted:** For highly sensitive columns (e.g. payment details), use Always Encrypted. The client driver encrypts data before sending it, and the server never sees plaintext ⁴. Keys are stored outside SQL (e.g. Azure Key Vault). This prevents even DBAs from reading encrypted columns.
- **Row-Level Security (RLS):** Implement RLS policies so users only see rows they should (e.g. a customer only sees their own orders) ⁵. Create security predicates with `CREATE SECURITY POLICY` and inline functions to filter.
- **Permissions and Roles:** Use Windows Authentication. Grant least privileges: e.g. create roles `dbo_Admin`, `Sales_Manager`, etc. Don't use `sa` for routine tasks.
- **Other protections:** Enable **TLS/SSL** for connections, disable unnecessary surface area. Use strong passwords and audit failed logins.

ETL & SSIS Approach

For initial bulk import (e.g. product catalog, past orders), use SQL Server Integration Services (SSIS):

- **Data Sources:** CSV or Excel from reputable datasets (e.g. UCI Online Retail) or extracted from a legacy system.

- **Data Flows:** SSIS Data Flow tasks can import **Product**, **Category**, **Supplier**, **Customer**, and **HistoricalOrders** tables. For example, map OnlineRetail.xlsx into staging tables, then transform/integrate into ShopEase schema.

- **Sample Package Outline:**

1. **Flat File Connections:** To CSV of orders and products.
2. **Data Conversion:** To match SQL types (date, numeric).
3. **Lookup/Upsert:** Use Lookup transformations to map product SKUs to IDs, or update existing records.
4. **OLE DB Destinations:** Insert into target tables.
5. **Error Handling:** Redirect rows on failure for cleansing.

Note: AdventureWorks sample data can be used for testing; e.g., its `Sales.SalesOrderHeader` table can populate Order data.

Reporting and SSRS

- **SSRS Reports:** Design reports for business users. Examples: *Daily Sales by Product*, *Inventory Levels*, *Customer Sales by Region*. Each report uses SQL queries or views (as above) as data sources.
- **Sample Report Query:** E.g. total sales per category:

```
SELECT c.Name AS Category, SUM(oi.Quantity*oi.UnitPrice) AS TotalSales
FROM OrderItem oi
JOIN Product p ON oi.ProductID=p.ProductID
JOIN Category c ON p.CategoryID=c.CategoryID
GROUP BY c.Name;
```

This can feed a chart in SSRS.

- **Dashboard Ideas:** Number of orders today, top 10 customers, stock alerts list (data-driven subscriptions possible). Use parameters for date range or category.

Testing Plan and Validation

- **Unit Testing:** Test each stored proc, trigger, and function with edge cases. E.g., try placing an order exceeding stock to see if trigger blocks it.
- **Data Validation:** Use CHECK constraints or additional queries: e.g., validate no negative quantities, all OrderItems link to valid Orders, email format in Customer.
- **Integration Testing:** Simulate workflows: create orders, cancel them, check inventory. Ensure triggers and procs maintain consistency.
- **Performance Testing:** Load large data sets (e.g. 1M orders) to test indexing and partition strategies. Use Query Store to compare plans.
- **Backup/Restore Test:** Regularly restore a backup to a test server to verify backups are usable.
- **Security Testing:** Attempt unauthorized access (e.g. SQL injection against test DB) and confirm encryption policies are in effect (cannot read encrypted data).

Implementation Timeline (10 Weeks)

```

timeline
    title 10-Week Implementation Plan
    Week 1 : Requirements review, sample data gathering
    Week 2 : ER diagram finalization, schema design
    Week 3 : Build tables (DDL) and import base data (SSIS)
    Week 4 : Implement core procedures, functions
    Week 5 : Develop triggers and views
    Week 6 : Index tuning, partition setup, columnstore load
    Week 7 : Security setup (TDE, AE, RLS), role configuration
    Week 8 : Develop SSRS reports, final queries
    Week 9 : Testing and data validation
    Week 10: Backup/restore testing, documentation, presentation prep

```

Each week has clear deliverables, ensuring an orderly build-up from data model to deployment and testing.

Presentation Outline

- **Slide 1: Title** – *ShopEase™ Advanced DBMS*, Team, Date.
- **Slide 2: Agenda** – Project overview, architecture, key features, security, performance, demo.
- **Slide 3: Objectives** – Project goals and scope.
- **Slide 4: ER Diagram** – Visual of schema (showing tables/relations from above mermaid).
- **Slide 5: Tables & Data Model** – List of main tables and purpose (with sample entity table).
- **Slide 6: Key Functions** – Brief on procedures/views/triggers (bullet points).
- **Slide 7: Sample Queries/Reports** – Screenshots or code of example analytics.
- **Slide 8: Indexing & Performance** – Outline columnstore usage, partitions, Query Store screenshots.
- **Slide 9: Security** – Summarize TDE, Always Encrypted, RLS (with citations noted).
- **Slide 10: Testing Results** – Show that constraints hold (e.g. negative stock prevented).
- **Slide 11: Timeline & Roles** – Mermaid timeline graphic, tasks per week.
- **Slide 12: Demo Steps** – Bullet list of demo flow (e.g. “Place order, run sales report”).
- **Slide 13: Challenges & Lessons** – Any significant issues encountered.
- **Slide 14: Conclusion** – Summary and Q&A.

A **demo script** might open SSMS: show creating a new order via `sp_CreateOrder`, show updated inventory, run an SSRS report, and illustrate encryption by querying an encrypted column (error for unauthorized user).

Summary of Key Items (Tables)

Entity	Key Attributes / Notes
Category	CategoryID (PK), Name, Description
Supplier	SupplierID (PK), Name, ContactInfo

Entity	Key Attributes / Notes		
Product	ProductID (PK), SKU, Name, Price, CategoryID→Category, SupplierID→Supplier		
Customer	CustomerID (PK), Name, Email, Phone		
Order	OrderID (PK), CustomerID→Customer, OrderDate, TotalAmount		
OrderItem	OrderItemID (PK), OrderID→Order, ProductID→Product, Quantity, UnitPrice		
Inventory	InventoryID (PK), ProductID→Product (unique), QuantityOnHand		
Index / Strategy	On Table (Columns)	Type	Notes
PK_Clustered	Category (CategoryID)	Clustered	Implied on each PK
PK_Clustered	Supplier (SupplierID)	Clustered	
PK_Clustered	Product (ProductID)	Clustered	
IX_Product_Category	Product (CategoryID)	Nonclustered	Filter by category
IX_Product_Supplier	Product (SupplierID)	Nonclustered	Filter by supplier
PK_Clustered	Customer (CustomerID)	Clustered	
PK_Clustered	Order (OrderID)	Clustered	
IX_Order_Customer	Order (CustomerID)	Nonclustered	Speeds customer order lookups
IX_Order_OrderDate	Order (OrderDate)	Nonclustered	For time-range queries
PK_Clustered	OrderItem (OrderItemID)	Clustered	
IX_OrderItem_Order	OrderItem (OrderID)	Nonclustered	Link to Order (batch retrieval)
IX_OrderItem_Product	OrderItem (ProductID)	Nonclustered	Link to Product
PK_Clustered	Inventory (InventoryID)	Clustered	
IX_Inventory_Product	Inventory (ProductID)	Nonclustered	Lookup inventory by product
Columnstore (Ex)	OrderItem (all columns)	Clustered Columnstore	High-compression analytics index ²
Sample Query		Purpose	
SELECT * FROM vw_DailySales		Fetch daily totals report	
SELECT * FROM vw_TopProducts		List top-selling products	

Sample Query	Purpose
SELECT * FROM Customer WHERE Email=?	Lookup customer by email (login scenario)
SELECT * FROM Product WHERE Name LIKE '%book%'	Search products by name
SELECT fn_TotalStockValue()	Get total inventory value (custom function)
SELECT * FROM [Order] WHERE OrderDate >= '2026-01-01'	Recent orders (operational)
SELECT COUNT(*) FROM OrderItem WHERE ProductID=3	Sales count for product 3
SELECT SUM(TotalAmount) FROM [Order] WHERE CustomerID=1	Lifetime spend of customer 1
SELECT * FROM Inventory WHERE QuantityOnHand < 10	Low stock alert (operational)
SELECT * FROM [Order] WHERE TotalAmount=0	Cancelled orders (after trigger action)
SELECT ... GROUP BY Category (join)	Revenue by category (for dashboard)
SELECT ... GROUP BY MONTH(OrderDate)	Monthly sales trend (power BI or SSRS chart)

Best-Practice Checklists

- **Normalization:** Ensure 1NF (atomic columns, PK), 2NF (no partial deps), 3NF (no transitive deps) ⁶ ⁷ . E.g. keep only product ID in OrderItem, not product name.
- **Indexing:** Only add indexes that benefit queries ¹ . Use include columns for covering indexes. Regularly rebuild and update stats. Consider filtered indexes for sparse queries. Use columnstore for large analytics tables ² .
- **Security:** Use Windows auth or Azure AD; use roles to limit rights. Encrypt sensitive columns (Always Encrypted) and files (TDE) ³ ⁴ . Implement RLS policies to restrict row access ⁵ .
- **Deployment:** Automate schema and data deployment via scripts or DACPACs. Use schema-binding for views/functions when possible to ensure consistency. Backup certificate keys (for TDE/AES) and test restores regularly.

Sources: Microsoft SQL Server documentation was referenced heavily (encryption features ³ ⁴ , security ⁵ , columnstore indexes ² , indexing guide ¹). AdventureWorks and public datasets (UCI Online Retail ⁸ ⁹) guided data modeling. All code and design choices follow best practices from MS Learn and proven e-commerce schema patterns.

¹ Index Architecture and Design Guide - SQL Server | Microsoft Learn

<https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-index-design-guide?view=sql-server-ver17>

² Columnstore indexes - Query performance - SQL Server | Microsoft Learn

<https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-query-performance?view=sql-server-ver17>

3 Transparent Data Encryption (TDE) - SQL Server | Microsoft Learn

<https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/transparent-data-encryption?view=sql-server-ver17>

4 Always Encrypted - SQL Server | Microsoft Learn

<https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine?view=sql-server-ver17>

5 Row-Level Security - SQL Server | Microsoft Learn

<https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security?view=sql-server-ver17>

6 7 Database Normalization – Normal Forms 1nf 2nf 3nf Table Examples

<https://www.freecodecamp.org/news/database-normalization-1nf-2nf-3nf-table-examples/>

8 9 UCI Machine Learning Repository

<https://archive.ics.uci.edu/dataset/352/online+retail>